Neural Networks

ICT 3.3 Machine learning and Data Mining

Emanuela Boros, May 2026

Neural Networks

So far, we’ve seen simple models like regression and classification.

But many real-world problems are much more complex.

This is where **neural networks** come in.

Why Do We Need Neural Networks?

- Real-world data is complex
- Linear models are limited
- Many patterns are non-linear
- We need more flexible models

From Linear to Non-Linear

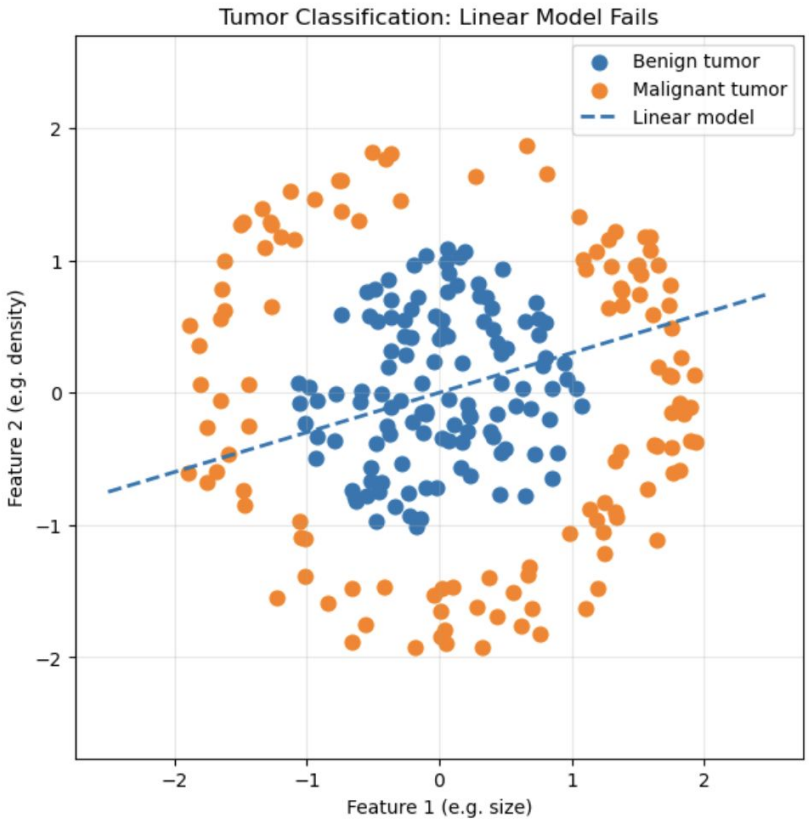
Do you think a straight line or a sigmoid can separate all types of data?

From Linear to Non-Linear

- Linear models → straight lines / simple boundaries
- Real data → complex patterns

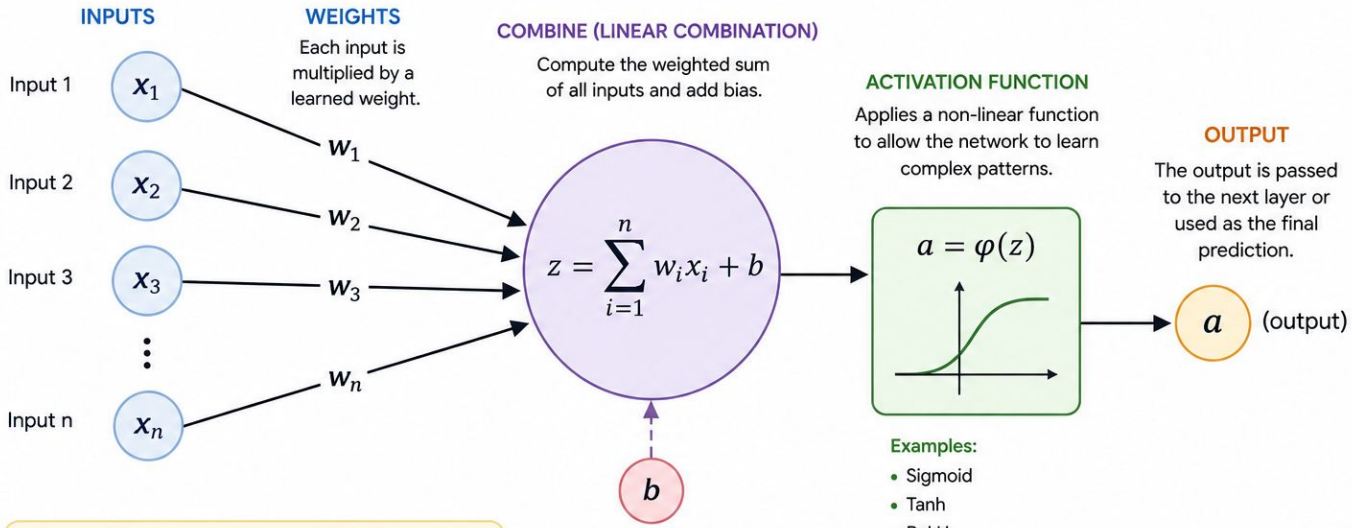
Imagine trying to separate two classes that form a circle.

→ non-linear transformations → **Neural networks** achieve this by combining many simple units to create complex behavior.



A Neuron in a Neural Network

Receives inputs, combines them, applies a non-linear activation, and produces an output.



WHAT THIS NEURON DOES

1. Receives inputs: $x_1, x_2, \ldots, x_n$
2. Multiplies each by a weight: $w_1, w_2, \ldots, w_n$
3. Sums them up and adds bias: $z = \sum_{i=1}^{n} w_i x_i + b$
4. Applies a non-linear activation: $a = \varphi(z)$
5. Produces output: a

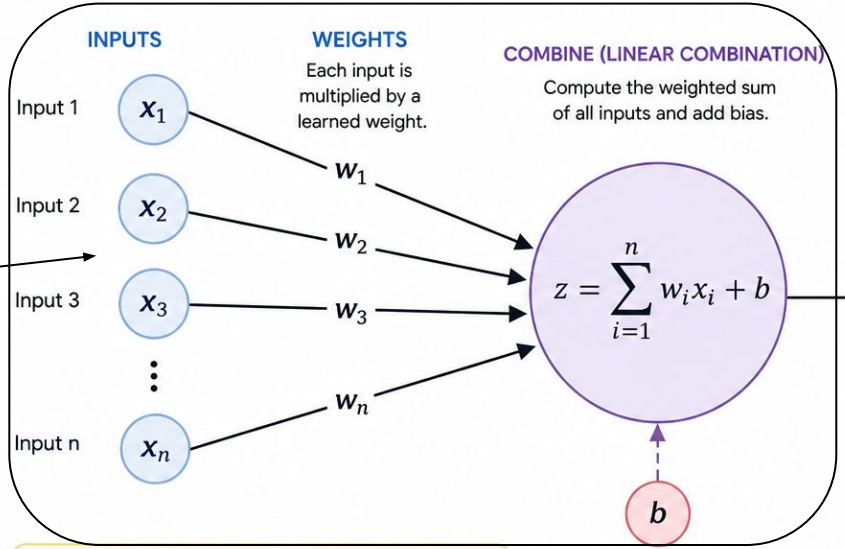
KEY IDEA

The activation function $\varphi(\cdot)$ is what makes neural networks powerful — it introduces non-linearity, enabling the network to model complex relationships.

A Neuron in a Neural Network

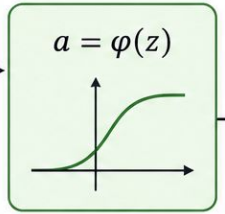
Receives inputs, combines them, applies a non-linear activation, and produces an output.

Does this remind you of something you learned in this course?



ACTIVATION FUNCTION

Applies a non-linear function to allow the network to learn complex patterns.



Examples:

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- ...

OUTPUT

The output is passed to the next layer or used as the final prediction.

a (output)

WHAT THIS NEURON DOES

1. Receives inputs: $x_1, x_2, \ldots, x_n$
2. Multiplies each by a weight: $w_1, w_2, \ldots, w_n$
3. Sums them up and adds bias: $z = \sum_{i=1}^{n} w_i x_i + b$
4. Applies a non-linear activation: $a = \varphi(z)$
5. Produces output: a

KEY IDEA

The activation function $\varphi(\cdot)$ is what makes neural networks powerful — it introduces non-linearity, enabling the network to model complex relationships.

What is a Non-Linear Function?

A non-linear function is a function that is not a straight line.

$$a = \varphi(z)$$

$$z = wx + b \rightarrow \text{linear part}$$

$$\varphi(z) \rightarrow \text{non-linear activation}$$

$$f(x) = ax + b$$

$$f(x) = x^2$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$f(x) = \max(0, x)$$

What is a Non-Linear Function?

A non-linear function is a function that is not a straight line.

$$a = \varphi(z)$$

$$z = wx + b \rightarrow \text{linear part}$$

$$\varphi(z) \rightarrow \text{non-linear activation}$$

$$f(x) = ax + b$$

$$f(x) = x^2$$

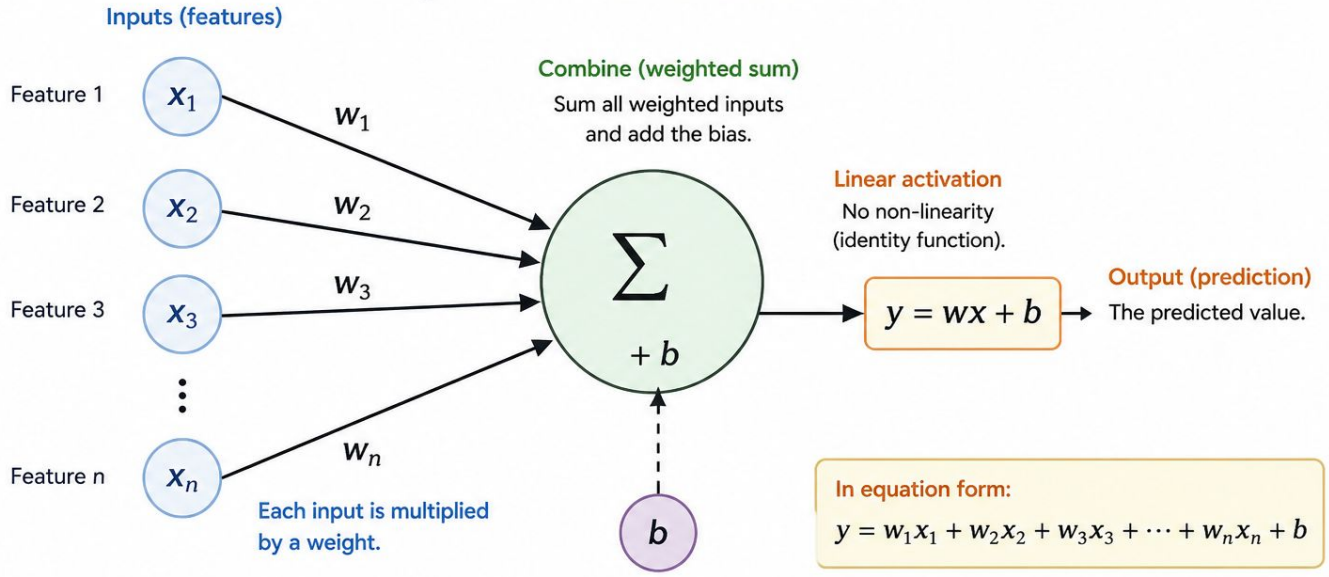
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$f(x) = \max(0, x)$$

ReLU (rectified linear unit)

Linear Regression as a Neuron

A single neuron with a linear activation function



In equation form:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n + b$$

x_i : inputs (features)

w_i : weights learned by the model

b : bias (intercept)

y : output (prediction)

Key idea

Linear regression is equivalent to a single neuron that computes a weighted sum of inputs plus a bias, with a linear activation function.

Neural Network

A **neuron** is essentially computing a **weighted sum of inputs** → a group of neurons form a **layer** → and a group of layers form …

Layers

A **neuron** is essentially computing a **weighted sum of inputs** → a group of neurons form a **layer**

- Input layer
- Hidden layers
- Output layer

Each layer builds on the previous one. **Early** layers learn **simple** patterns… **Later** layers combine them into more **complex** patterns.

A neural network is just a collection of neurons arranged in layers.

Input Layer

The input layer receives the raw data

Example

- image → pixels
- house → size, rooms, location
- text → word features

→ It does **no learning** just passes information forward

Hidden Layers

Layers where the model learns patterns

- combine inputs
- apply weights + activation
- extract features

Learning process

- early layers → simple patterns
- deeper layers → complex patterns

Purpose

transform raw data into useful representations → learn how to represent the data

Example:

- layer 1 → edges
- layer 2 → shapes
- layer 3 → objects

Output Layer

Produces the final prediction

- regression → one number
- binary classification → probability
- multi-class → class probabilities

→ **translates learned features into a decision**

Neural Network

But stacking linear functions does **NOT** create complexity → to model complex patterns, we need something else…

Neural Network

But stacking linear functions does **NOT** create complexity → to model complex patterns, we need something else…

- We need non-linearity.
- This is exactly what **activation functions** provide.

Activation functions → filters or decision mechanisms

Reminder: without activation functions, neural networks would be no more powerful than linear models.

Neural Network - Activation Functions

There are many activation functions, but we'll focus on a few common ones.

- **Sigmoid**
  Output between 0 and 1
- **Softmax**
  Output between 0 and 1
- **ReLU** (Rectified Linear Unit)
  Output = max(0, x)
- **Tanh**
  Output between -1 and 1

Sigmoid

Sigmoid maps any value into a range between 0 and 1.

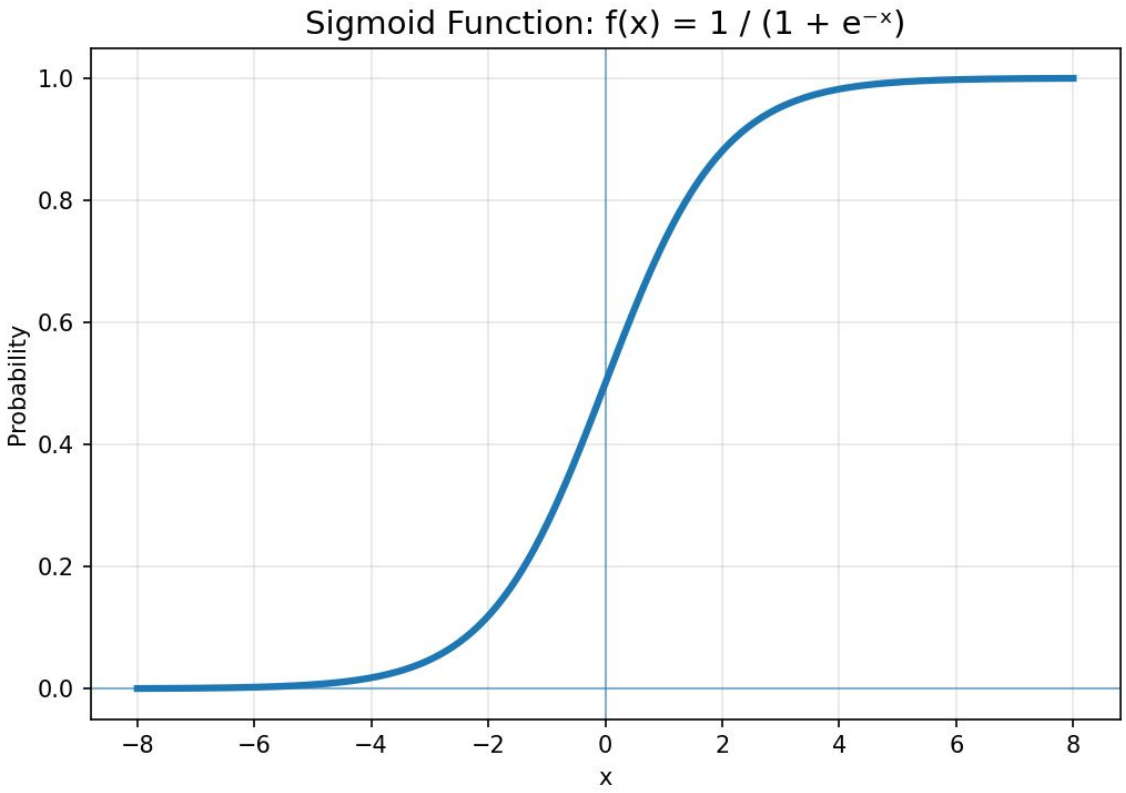
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- squashes any value into a probability
- small → close to 0
- large → close to 1

🎯 When to use

- binary classification output
- probability prediction

Sigmoid


Sigmoid Function: f(x) = 1 / (1 + e⁻ˣ)
Probability
1.0
0.8
0.6
0.4
0.2
0.0
−8
−6
−4
−2
0
2
4
6
8
x

ReLU

ReLU is very simple: ReLU keeps positive information and ignores the rest.

→ This is the most widely used activation function today.

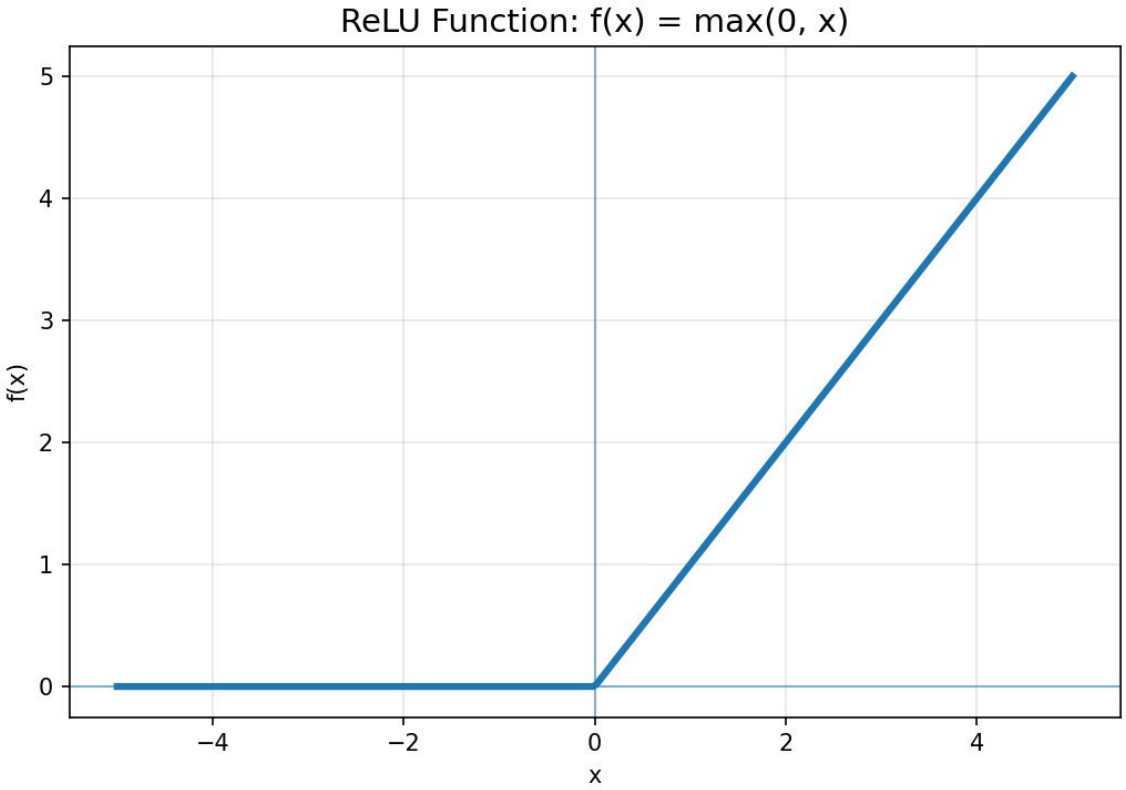
$$\mathrm{ReLU}(x) = \max(0, x)$$

- negative values → 0
- positive values → unchanged

🎯 When to use

- hidden layers
- (really) deep neural networks

ReLU



Tanh

Tanh is similar to sigmoid, but ranges from -1 to 1.

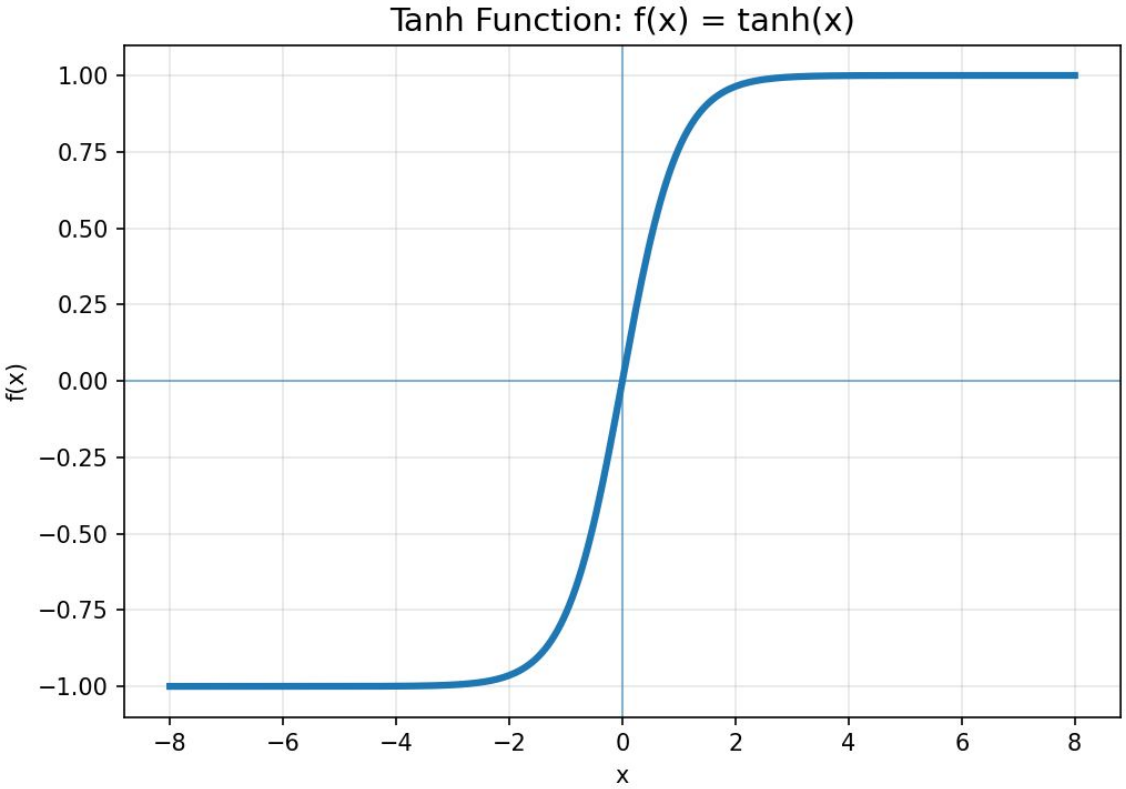
$$\tanh(x)$$

- similar to sigmoid but centered at 0
- negative → negative
- positive → positive

🎯 When to use

- hidden layers (sometimes)
- when data is centered

Tanh


Tanh Function: f(x) = tanh(x)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
f(x)
−8
−6
−4
−2
0
2
4
6
8
x

Softmax

Softmax:

- takes multiple values (scores/logits)
- turns them into a probability distribution

→ the largest value → highest probability

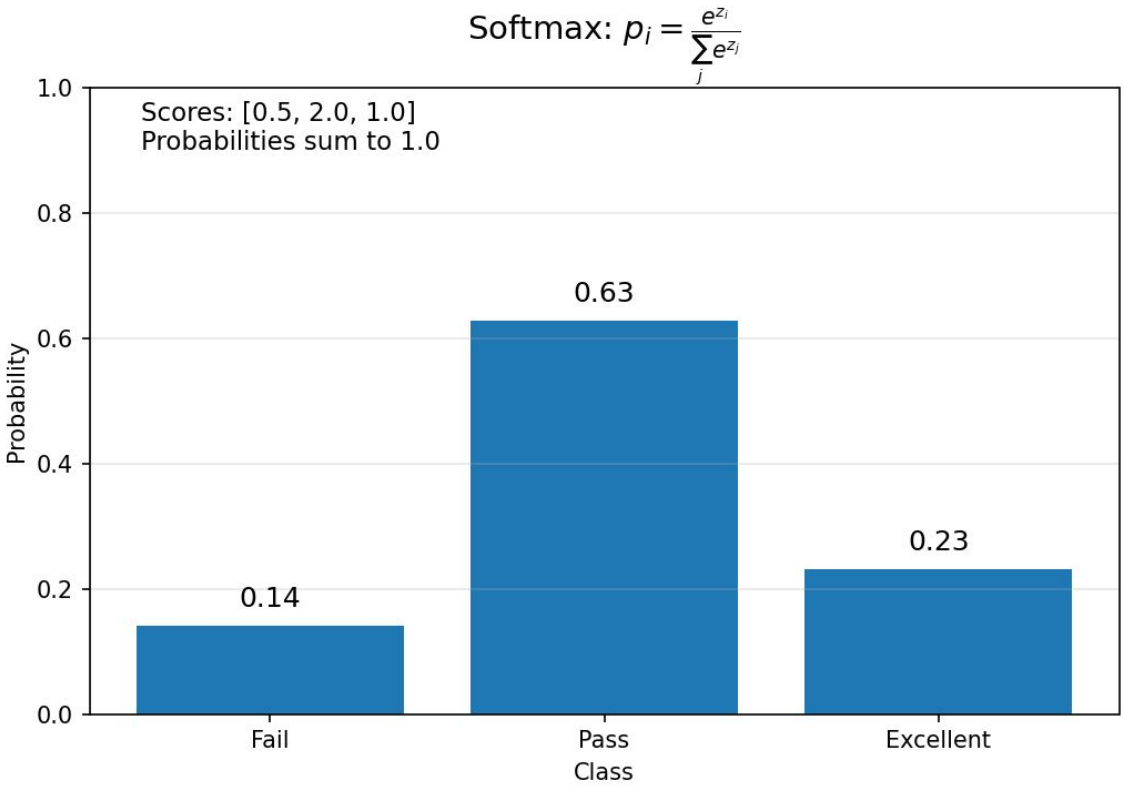
🎯 When to use

- used only in output layer
- multi-class classification output
- when classes are mutually exclusive

$$\mathrm{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

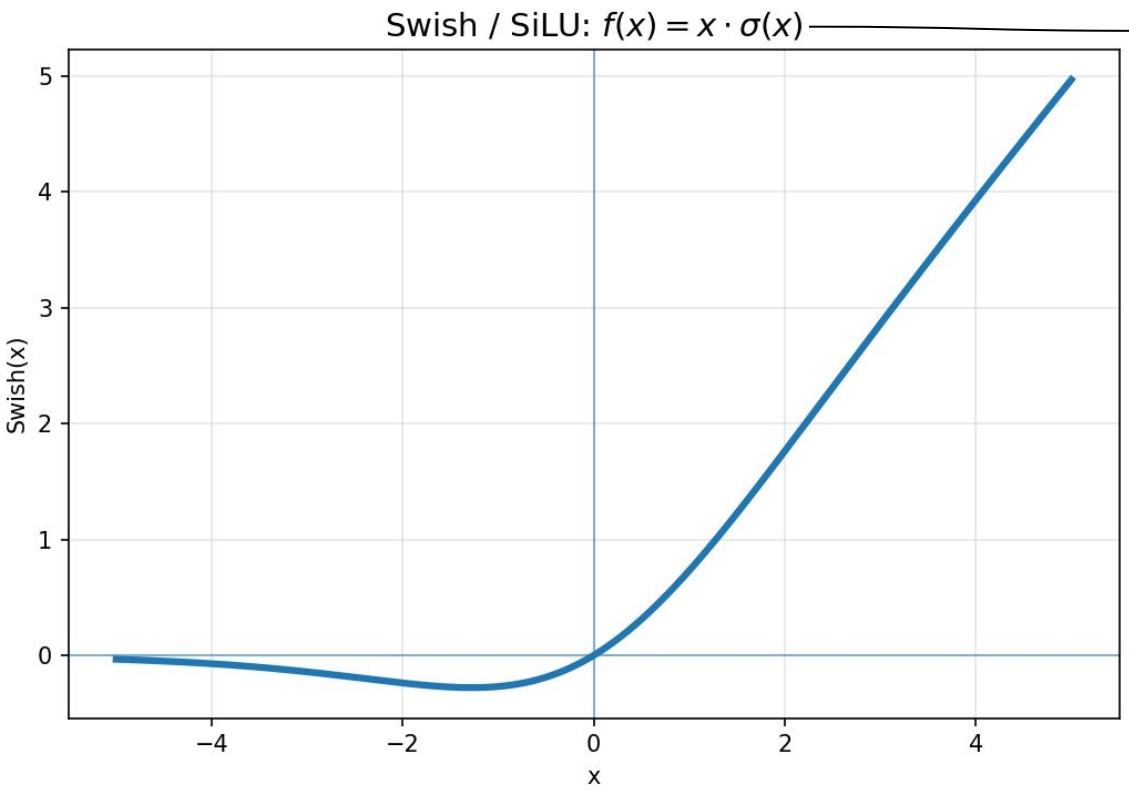
The probabilities add up to 1!

Softmax


Softmax: $p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$
Scores: [0.5, 2.0, 1.0]
Probabilities sum to 1.0
1.0
0.8
0.6
0.4
0.2
0.0
Probability
0.14
0.63
0.23
Fail
Pass
Excellent
Class

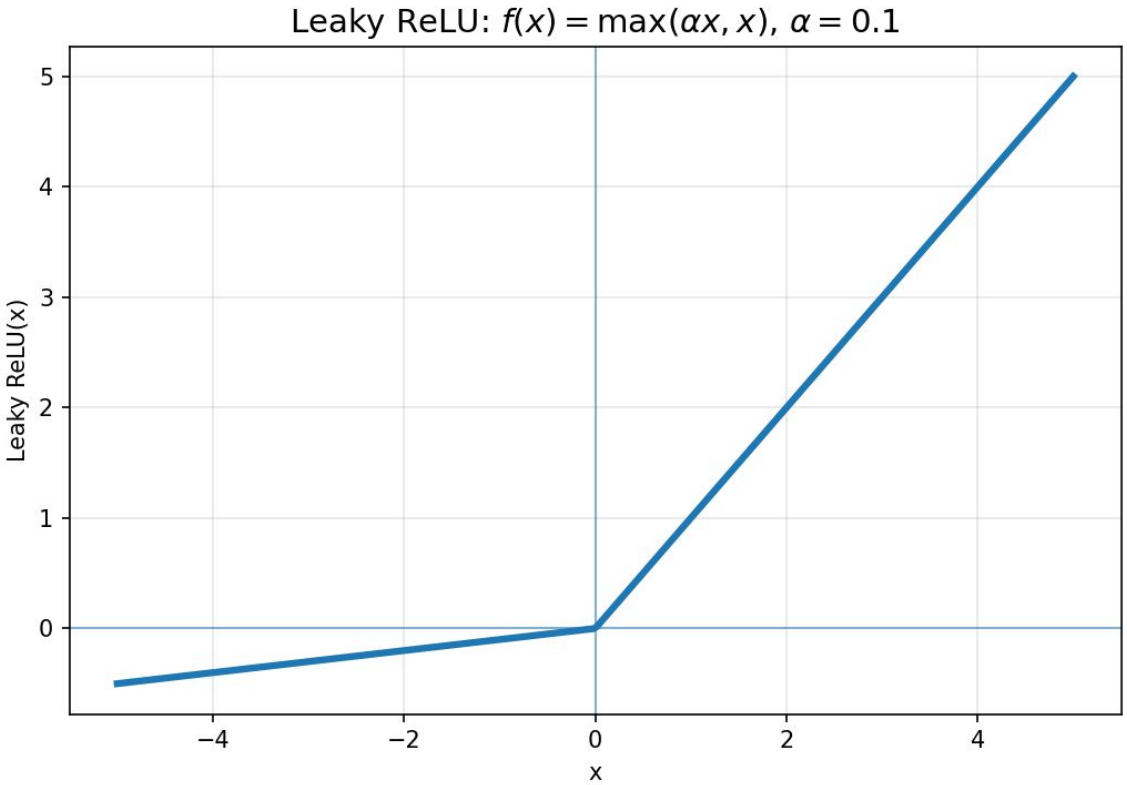
Others

Swish/SiLU - Sigmoid Linear Unit (SiLU)



σ(x) is the logistic sigmoid.

Leaky ReLU


Leaky ReLU: $f(x) = \max(\alpha x, x)$, $\alpha = 0.1$
Leaky ReLU(x)
x
5
4
3
2
1
0
−4
−2
0
2
4

Activation Functions by Layer

- **Input** Layer → Just passes raw data → No activation
- **Hidden** Layers → Learn patterns and features → Usually nonlinear activations

Common **hidden**-layer activations:

- *ReLU* → most common default choice for hidden layers
- *Tanh* → older/simple networks, outputs values between -1 and 1
- *Leaky ReLU* → like ReLU, but avoids completely “dead” neurons (link)
- *Swish / SiLU* → smooth ReLU-like function used in some modern deep networks

Activation Functions by Layer

Output Layer → Depends on the task

- **Sigmoid** → binary classification
  - Example: not successful / successful
  - Output: one probability between 0 and 1
- **Softmax** → multi-class classification
  - Example: fail / pass / excellent
  - Output: one probability per class, all probabilities sum to 1
- Linear / Identity → regression
  - Example: predict grade, price, temperature
  - Output: any real number
- **ReLU** → sometimes used in regression when the output must be non-negative
  - Example: predict duration, count, or price when negative values make no sense

Activation Functions by Layer

Rule to remember:

- **Hidden** layers learn representations.
- **Output** layers produce the final answer.
- ReLU, Tanh, GELU, Swish → usually used to learn inside the network.
- Sigmoid, Softmax, Linear → usually used at the end to match the task.

Activation Functions by Layer

And short version:

- **ReLU** learns.
- GELU/Swish learn in modern networks.
- **Sigmoid** decides between *two classes*.
- **Softmax** decides between *many classes*.

HOW DATA FLOWS THROUGH A NEURAL NETWORK

From raw input → linear transformations → nonlinear activations → final prediction

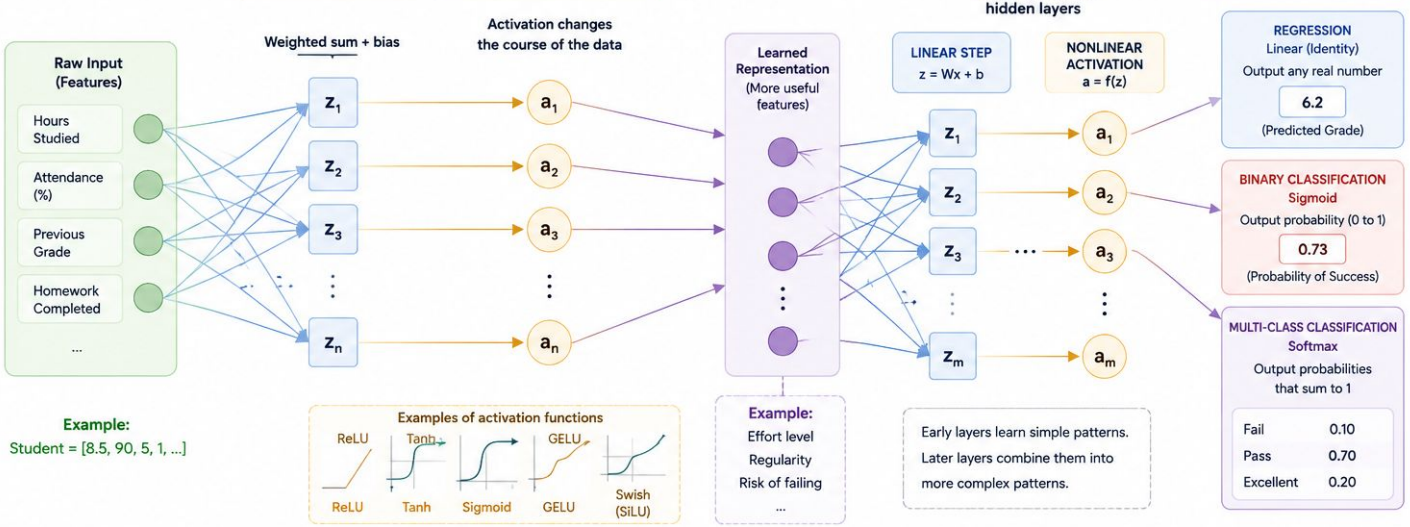
1. INPUT LAYER
Just passes raw data
No activation

2. LINEAR STER
Each neuron makes a linear calculation
$z = Wx + b$

3. NONLINEAR ACTIVATION
Apply activation function
$a = f(z)$
(Nonlinear)

4. NEW REPRESENTATION
The outputs become a new representation (features)

5. OUTPUT LAYER
Produces the final answer (Activation depends on the task)



Raw Data → **Linear Transformation** ($z = Wx + b$): Moves and stretches the data → **Nonlinear Activation** ($f(z)$): Bends the data and introduces nonlinearity → **New Representation**: More useful features for the next layer → ... → **Output / Prediction**: Final answer based on the task

Neural networks repeatedly apply a linear transformation and then a nonlinear activation, gradually transforming raw data into useful representations until the output layer produces the final prediction.

What does it mean that a layer “learns”?

z = wx + b → the model changes w and b to reduce the error → activation function transforms z: a = f(z)

The **activation** decides **how much signal passes to the next layer**.

ReLU:

```
ReLU(z) = max(0, z)
```

- If `z = -2 → ReLU(z) = 0`
- If `z = 3 → ReLU(z) = 3`

What does it mean that a layer “learns”?

It acts like a **gate**:

- negative signal → blocked
- positive signal → passed

The network learns by changing the weights so that useful signals become **active**.

What does it mean that a layer “learns”?

What does it mean when a `ReLU(z) = 0`?

How Do Neural Networks Learn?

How Do Neural Networks Learn?

- start with random weights
- make predictions
- measure error
- adjust weights

Training = **adjusting weights** to reduce error

Loss Function

A function that measures how wrong the model is.

- prediction ≠ true value → error
- loss tells us how big that error is

Types?

Loss Function

A function that measures how wrong the model is.

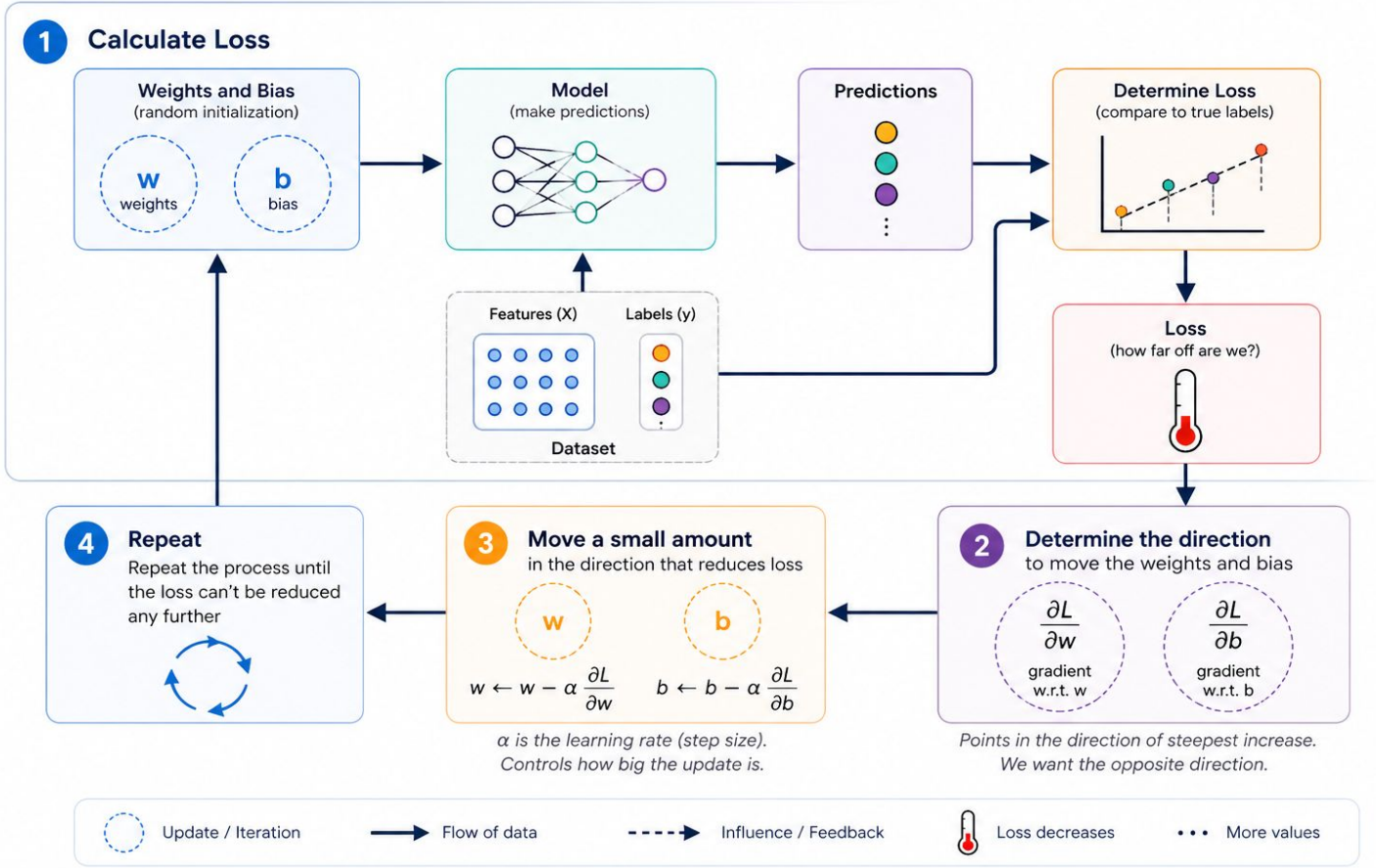
- prediction ≠ true value → error
- loss tells us how big that error is

Different types

- MSE → regression
- Cross-entropy → classification

GRADIENT DESCENT

Iteratively find the weights (w) and bias (b) that minimize the loss



GD

1 Predict
Model makes a prediction

→

2 Compute loss
Measure how wrong it is

→

3 Compute gradient
Find direction of change

→

4 Update weights
Move weights downhill

Update rule

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \nabla L$$

η = learning rate ∇L = gradient of the loss

Momentum

Remembers part of the previous direction, so updates move more smoothly instead of zigzagging.

→ walking downhill → rolling a ball downhill

Types of Gradient Descent

Type	Data used per update	
Batch GD	All training examples	Stable but slow
Stochastic GD (SGD)	1 example	Fast but noisy
Mini-batch GD	Small group, e.g. 32	Most common compromise

Training repeats this cycle until the loss becomes smaller.

Training a Neural Network

for each epoch:

for each batch:

→ **forward** pass → make predictions

→ compute **loss** → measure error

→ **backpropagation** → compute gradients

→ **update** weights → gradient descent step

We learn step by step, batch by batch, improving over time.

Training a Neural Network

- batches → small learning steps
- epochs → repetition
- training → gradual improvement

Key Challenges in ML

Overfitting: Model memorizes instead of learning

Underfitting: Model too simple

Data Quality: Garbage in → garbage out

Key Challenges in ML

Too few epochs → underfitting

Too many epochs → overfitting

Epochs

- One epoch = one pass over the entire dataset
- Training uses multiple epochs
- Model improves over time (check the loss on training and dev data)

Batches

- A batch = a small subset of the dataset
- Training is done batch by batch, not all at once
- Makes training faster and more stable
- Each batch → one update of the model

Epochs vs Batches

Epoch = full pass, Batch = small step inside it.

Training vs Test vs Dev Data

- **Training Data**
  Used to learn the model
- **Dev Data**

  Used to evaluate the model during training (check for over- or underfitting)
- **Test Data**
  Used to evaluate performance

Training vs Test vs Dev Data

Why shouldn’t we train on the test data?

Why Do Neural Networks Work Well?

- Can model complex, non-linear patterns
- Combine many simple units
- Learn hierarchical representations
- Scale with data

Neural Networks

Do you think neural networks would work well with very little data?

Limitations of Neural Networks

1 — Data

They usually require a lot of data to perform well.

2 — Computation

Training can be slow and resource-intensive.

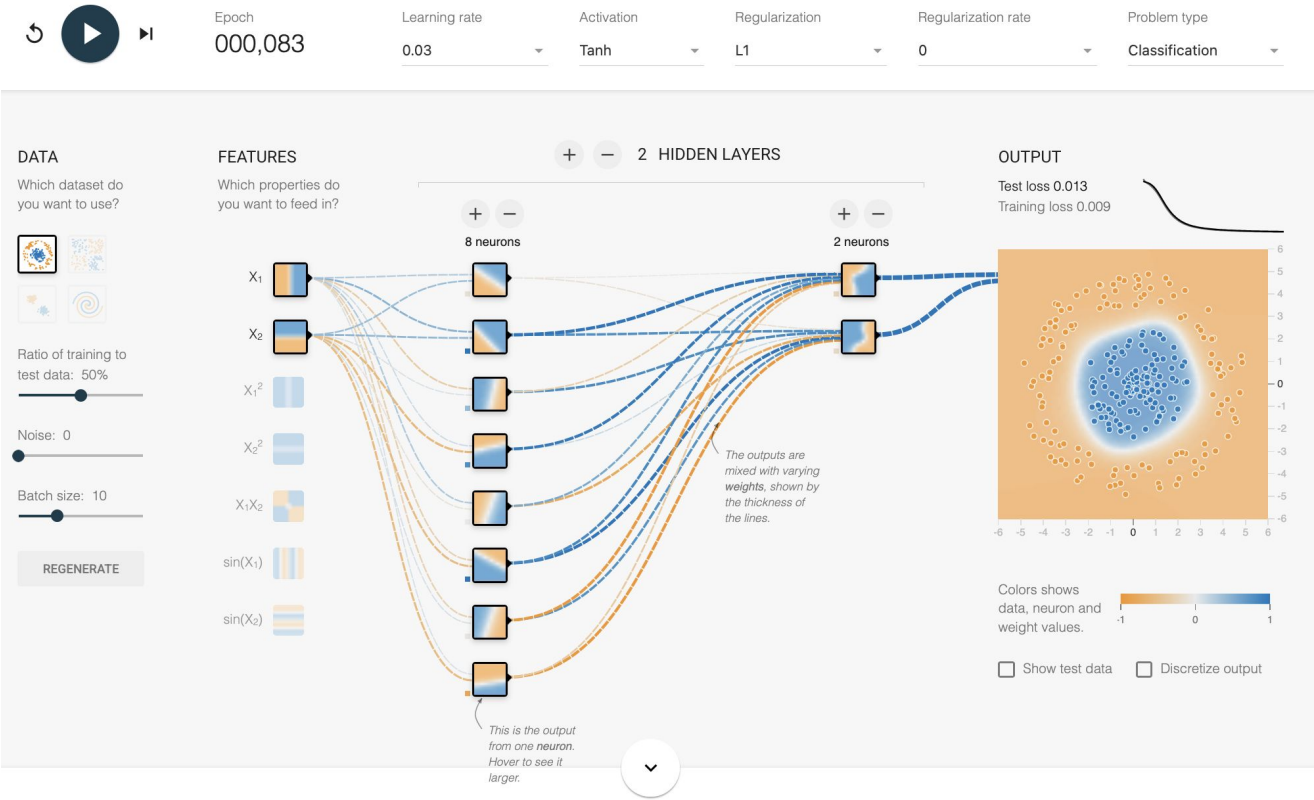
3 — Interpretability

It’s often difficult to understand *why* a model makes a decision.

4 — Overfitting

They can easily memorize data if not handled carefully.

https://playground.tensorflow.org/



Neural Networks

Neural networks are powerful tools, but they must be used carefully.

Which limitation do you think is most problematic in real-world applications?

Mini Assignment

TODOs:

- Exercise notebook

Clone the repo!